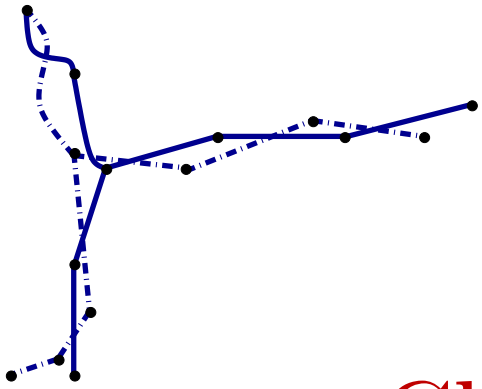
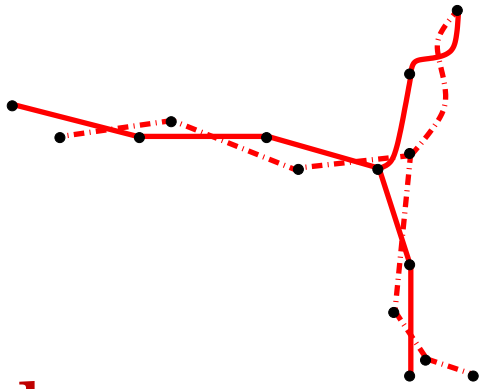
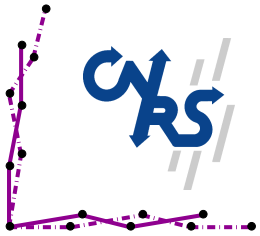




Cloud et Réseaux Virtuels

https://perso.lip6.fr/binh-minh.bui-xuan/crv2025.html

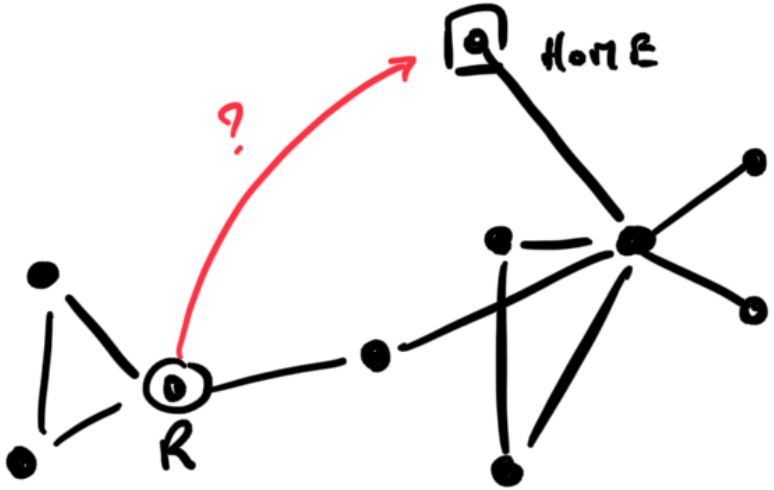
Binh-Minh Bui-Xuan



SORBONNE UNIVERSITÉ
CRÉATEURS DE FUTURS DEPUIS 1257

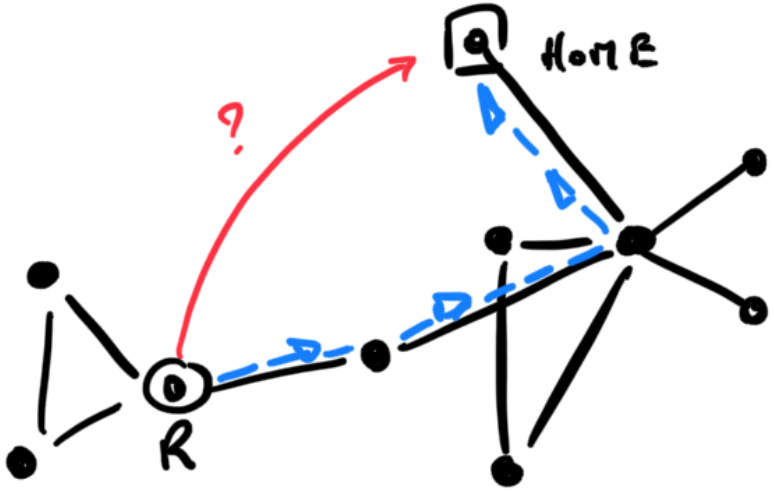
PARIS, April 2026

Robot navigation, with time?



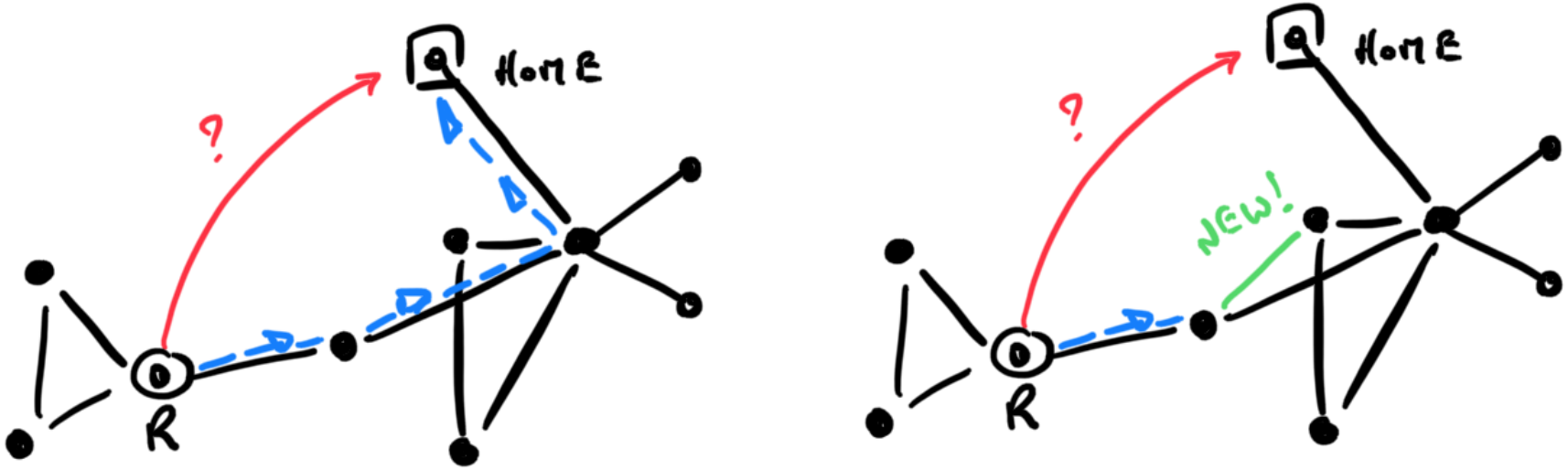
R: get HOME?

Robot navigation, with time?



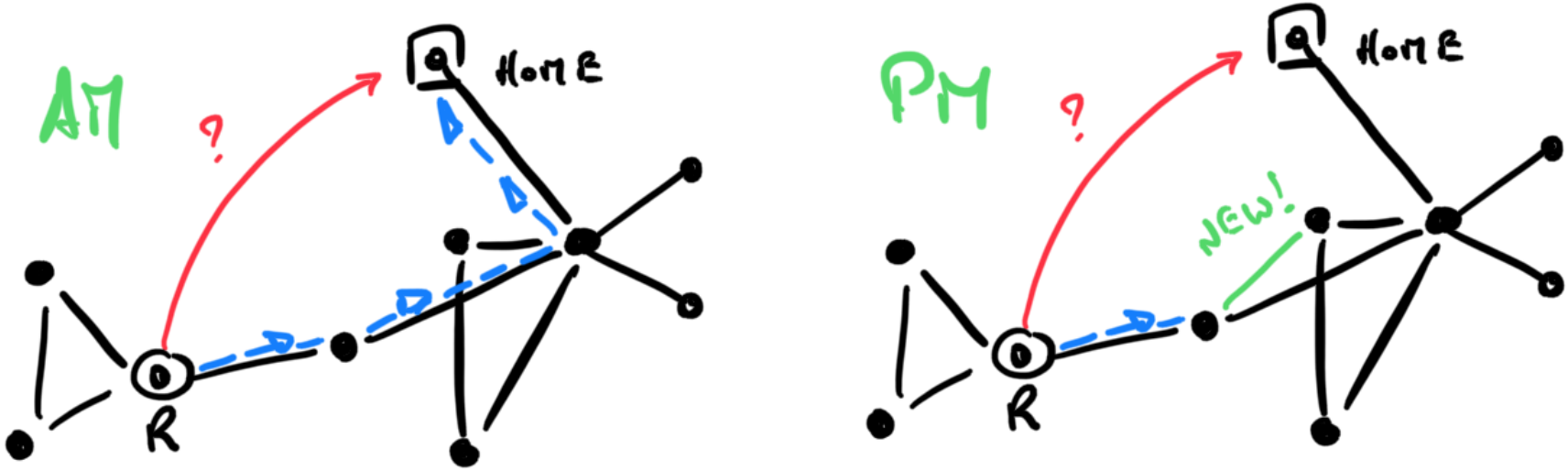
R: get HOME? Fast? (distance $\approx$ time)

Robot navigation, with time?



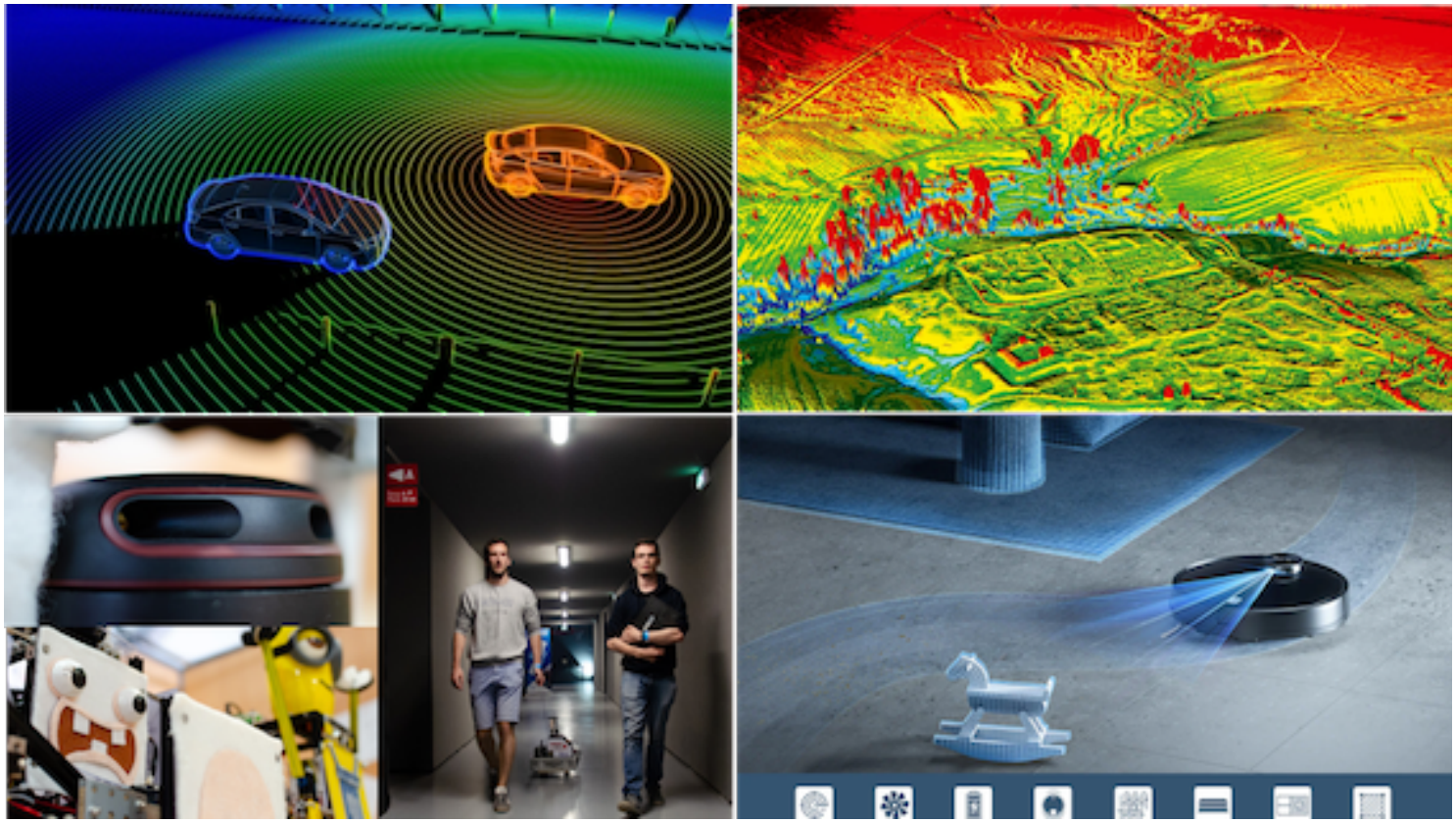
R: get HOME? Fast? Change route?

Robot navigation, with time?



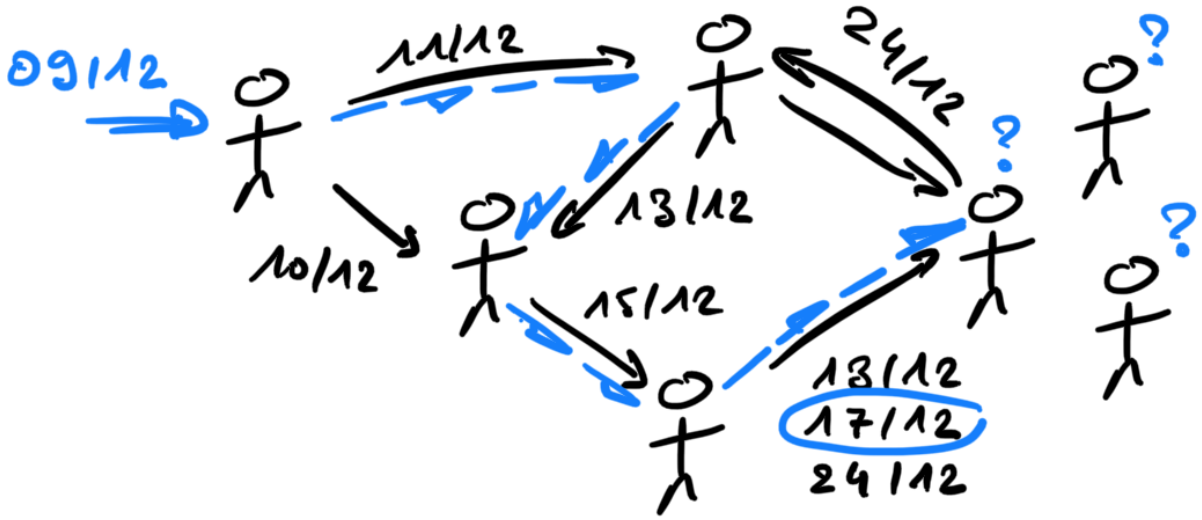
R: get HOME? Fast? Change route? To wait or not to wait?

Some motivations



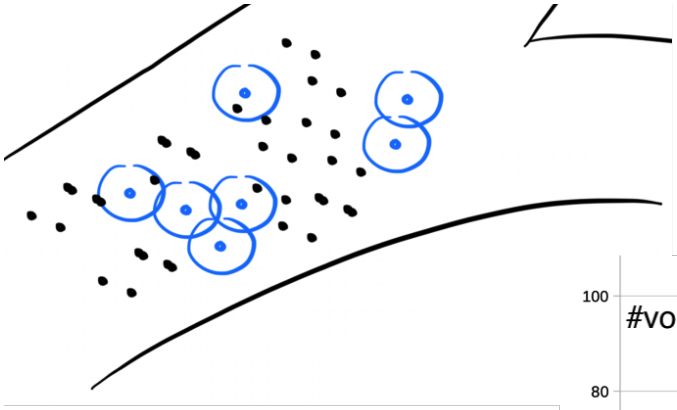
SLAM (Simultaneous Localization And Mapping).

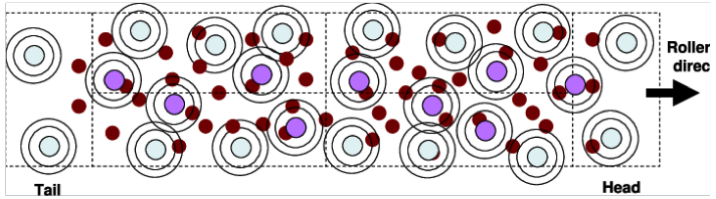
Some motivations

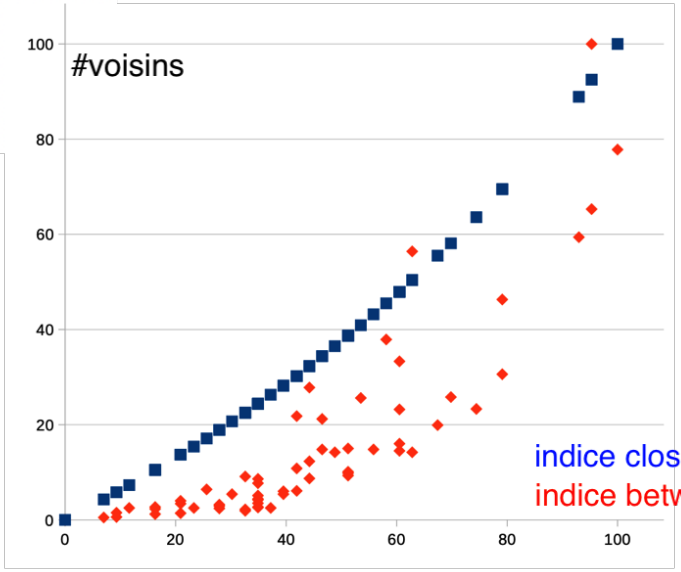


Viral propagation (rumor, ...).

Some motivations

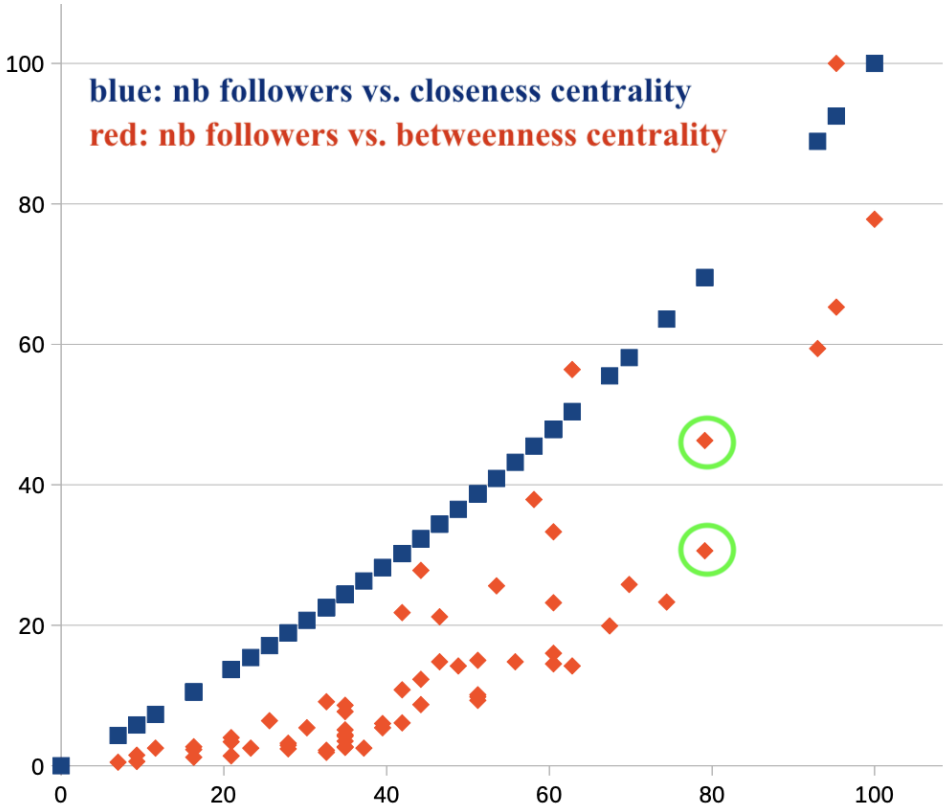




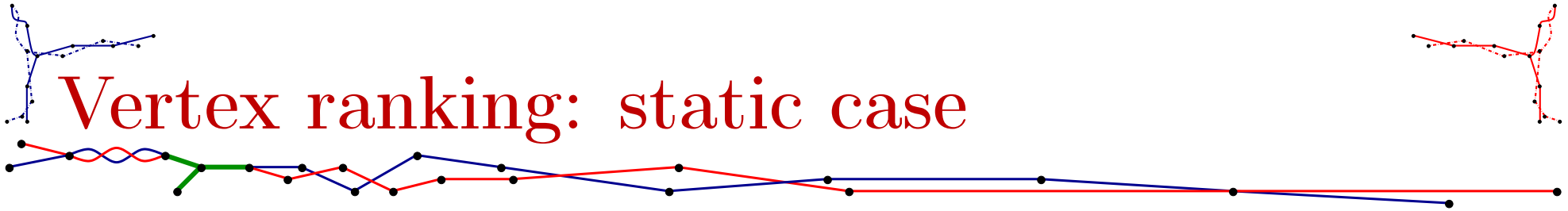


Vertex rankings (NoSEO, Not only Search Engine Optimisation).

Vertex ranking: static case



Rollernet 20m-30m (union graph): betweenness centrality $\neq$ nb. followers



Vertex ranking: static case

A path is a sequence of adjacent edges:

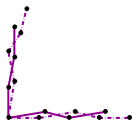
- "1-to-1" path: Dijkstra, good implementation exercise.
- "all-to-all" paths: Floyd-Warshall, simple implementation.

Closseness centrality:

- $c\_index(u) \approx \frac{1}{d(u,v_1)+d(u,v_2)+d(u,v_3)+d(u,v_4)+...}$
- for every line (or column) of Floyd-Warshall's output, sum all.

Betweenness centrality:

- $b\_index(u) \approx \frac{\#\text{optimal paths from } v_i \text{ to } v_j \text{ passing by } u}{\#\text{optimal paths from } v_i \text{ to } v_j}$
- good exercise: modifications to Floyd-Warshall or to BFS.



Vertex ranking: static case, implementation?

Desktop — vim -c set encoding=utf-8 closeness_and_betweenness.py — 143×35

```
cat 2030.nodup| awk '{degree[$1]++; degree[$2]++} END {for (vertex in degree) {print vertex " " degree[vertex]}}' > 2030.vertexDegree
~
~
```

nombre_de_voisins.sh 1,1 All

```
M = [[0]*n for i in range(n)]
dist = [[float('inf')]*n for i in range(n)]

# calcul de la matrice dist avec Floyd-Warshall
with open('2030.nodup', 'r') as inFile:
  for line in inFile.readlines():
    c1,c2=line.split()
    i=int(c1)
    j=int(c2)
    M[i][j]=1
    M[j][i]=1
    dist[i][j]=1
    dist[j][i]=1
  for i in range(n):
    dist[i][i]=0
  for k in range(n):
    for i in range(n):
      for j in range(n):
        if (dist[i][j]>dist[i][k]+dist[k][j]):
          dist[i][j]=dist[i][k]+dist[k][j]

# calcul du closeness centrality
cc=[0]*n
for i in range(n):
  denominateur=reduce(lambda x,y:x+y,dist[i])
  cc[i]=(n-1)/denominateur

with open('2030.closeness', 'w') as output:
  for i in range(n):
```

closeness.py 28,1 92%

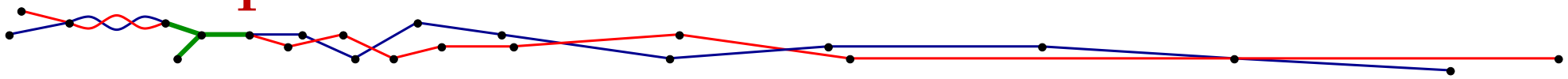
```
for k in range(n):
  for i in range(n):
    for j in range(i+1,n):
      if i!=k and k!=j and dist[i][j]>dist[i][k]+dist[k][j]:
        if DEBUG_ON:
          print(color.GREEN+"===IF====="+color.END)
          print("i k j",i,k,j)
          print("---START----")
          printall()
        dist[i][j]=dist[i][k]+dist[k][j]
        dist[j][i]=dist[i][j]
        nbPaths[i][j]=nbPaths[i][k]*nbPaths[k][j]
        nbPaths[j][i]=nbPaths[i][j]
        nbPathsStep[i][k][j]=nbPaths[i][j]
        nbPathsStep[j][k][i]=nbPathsStep[i][k][j]
        for iPrime in range(n):
          if nbPathsStep[i][iPrime][k]>0:
            nbPathsStep[i][iPrime][j]+=nbPathsStep[i][iPrime][k]*nbPaths[k][j]
            nbPathsStep[j][iPrime][i]=nbPathsStep[i][iPrime][j]
        for jPrime in range(n):
          if nbPathsStep[k][jPrime][j]>0:
            nbPathsStep[i][jPrime][j]+=nbPaths[i][k]*nbPathsStep[k][jPrime][j]
            nbPathsStep[j][jPrime][i]=nbPathsStep[i][jPrime][j]
        if DEBUG_ON:
          print("---END------")
          printall()

      elif i!=k and k!=j and dist[i][j]!=float('inf') and dist[i][j]==dist[i][k]+dist[k][j]:
        if DEBUG_ON:
```

closeness_and_betweenness.py 95,1 54%

Static case: computation of degree, closeness, and betweenness centrality.

Our questions



Needs:

- distance: optimal arrival time?
- route: journey (path) reconstruction?
- vertex ranking: number of optimal journeys?

Scenarios:

- journeys with stops: TCP/IP, bus, ...
- journeys without (long) stops: virus, rumor, aircrafts, ...
- journeys = temporal paths vs. temporal walks.
- graphs = geometric vs. general case.

Known cases

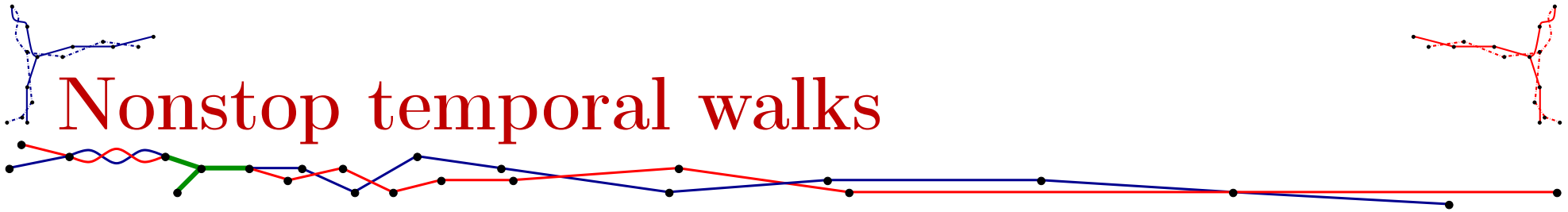
Need \ Scenario	temporal walks + stops temporal paths + stops	nonstop temporal walks	nonstop temporal paths
distance	polynomial [1]		NP-hard [2]
reconstruction	idem		idem
enumeration	idem		idem

- [1] BX., Ferreira, Jarry, 2003
- [2] Casteigts, Himmel, Molter, Zschoche, 2021

New results

Need \ Scenario	temporal walks + stops temporal paths + stops	nonstop temporal walks	nonstop temporal paths
distance	polynomial [1]	linear [3]	NP-hard [2]
reconstruction	idem	linear [4]	idem
enumeration	idem	open	idem

- [1] BX., Ferreira, Jarry, 2003
- [2] Casteigts, Himmel, Molter, Zschoche, 2021
- [3] Villacis-Llobet, BX., Potop-Butucaru, 2022
- [4] Brunelli, Viennot, 2023



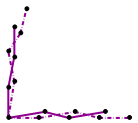
Nonstop temporal walks

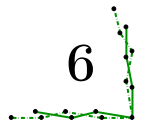
DEFINITION:

- ▷ Temporal graph $G = (\tau, V, A, c)$ is:
  - ◇ $\tau \in \mathbb{N}$ representing $T = [\![0, \tau - 1]\!]$, the datetime set
  - ◇ V a finite set, the vertex set
  - ◇ $A \subseteq T \times V \otimes V$, the arc set where $V \otimes V = V \times V \setminus \{(v, v) : v \in V\}$
  - ◇ $c : A \to \mathbb{N}$, the traversal time for every arc

- ▷ Non-stop journey (temporal walk) $J = (a_1, a_2, \ldots, a_p)$ is:
  - ◇ $a_i = (d_i, s_i, t_i) \in A$ for every $1 \leq i \leq p$
  - ◇ $t_i = s_{i+1}$ and $d_i + c(a_i) = d_{i+1}$ for every $1 \leq i \leq p - 1$
  - ◇ $d_p + c(a_p)$ is the arrival date of J

MAIN RESULT: *From input $G = (\tau, V, A, c)$ and $s, t \in V$, the minimum arrival date of a non-stop journey from s to t can be computed in linear time.*





Experiments (ongoing works)

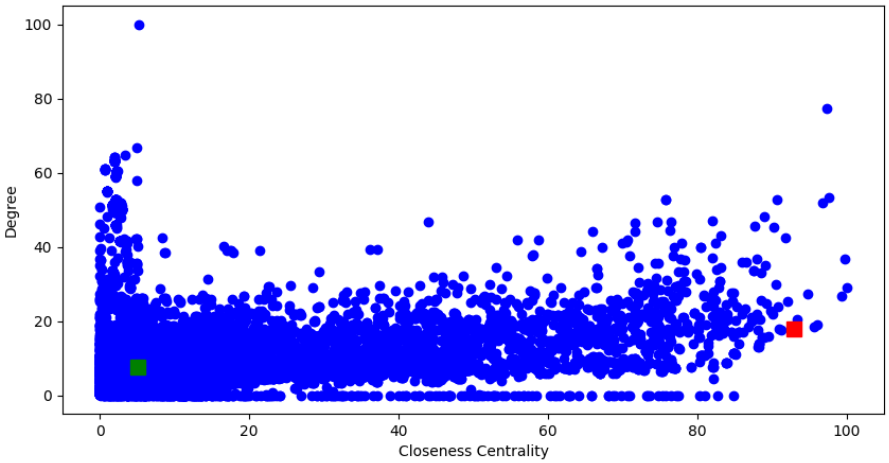
OUTLINES:

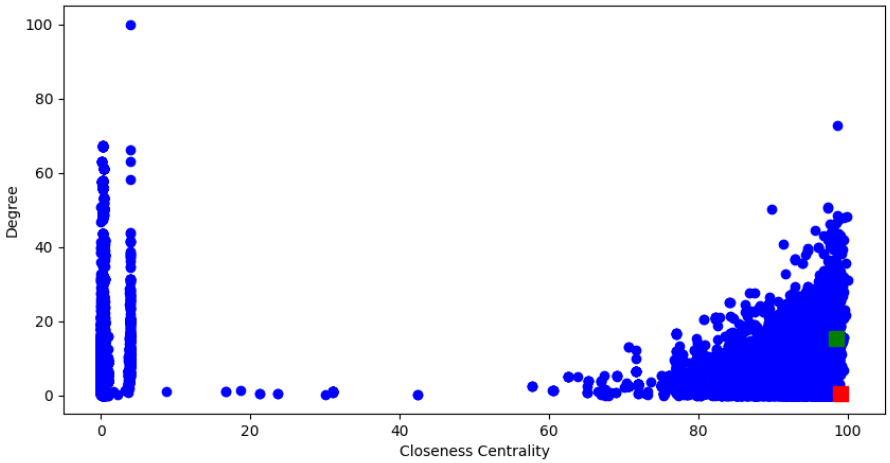
- ParisPT: $n = 12k, m \approx 2m, \tau \approx 1k$; directed.
- ParisPT PM: $n = 12k, m \approx 750k, \tau \approx 1k$; directed.

REMARKS:

- only for temporal closeness centrality (not betweenness).
- outgoing closeness centrality (cc^+) $\neq$ incoming closeness centrality (cc^-).

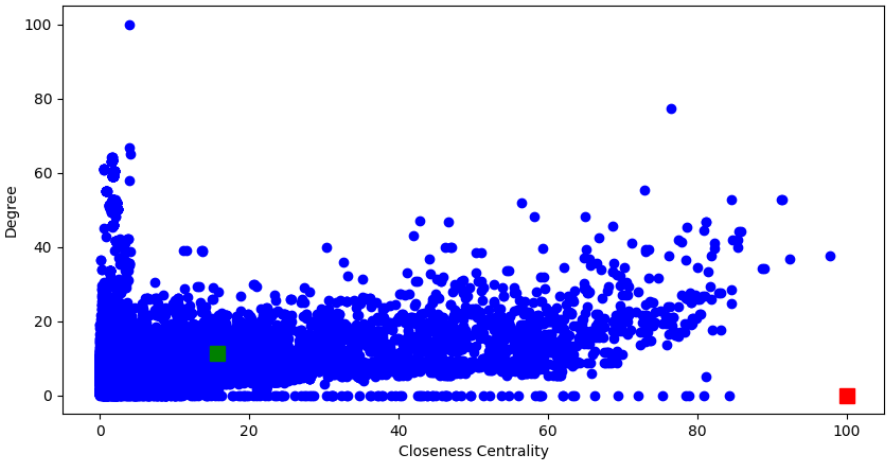
ParisPT vs. ParisPT PM: outbounds

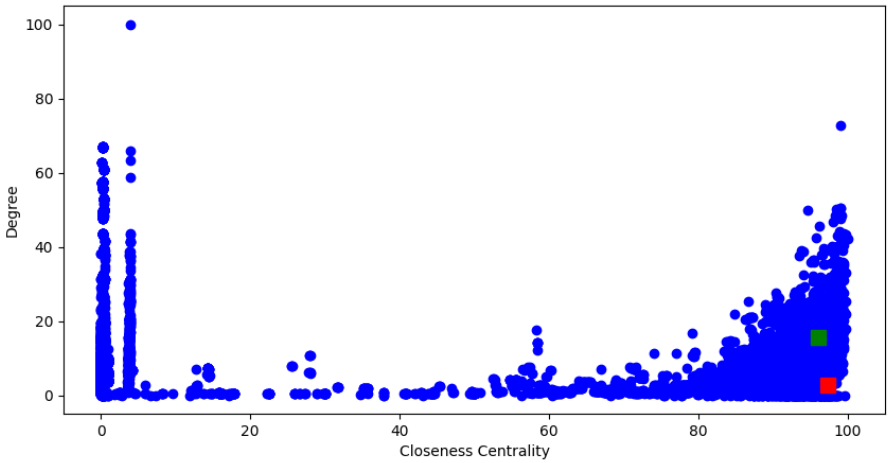




- Axis: cc^+ vs. δ^-.
- Nonstop constraint: less than 10min waits.
- Main result: Green dot is Hotel de Ville; red dot is Leblanc-Delbarre.

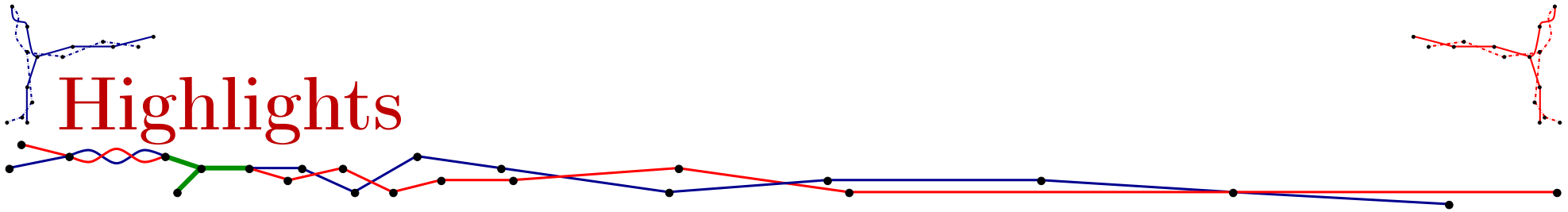
ParisPT vs. ParisPT PM: inbounds





- Axis: cc^- vs. δ^+.
- Nonstop constraint: less than 10min waits.
- Main result: Green dot is Gare du Nord; red dot is La Sibelle.

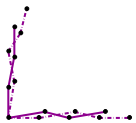
Highlights

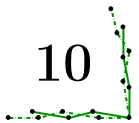


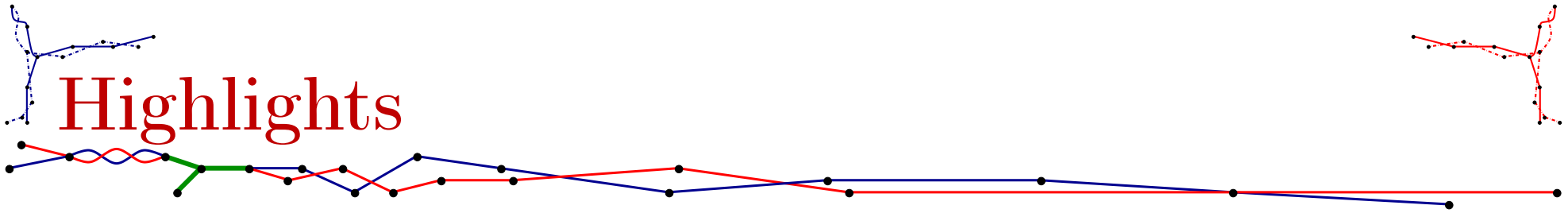
1. Static case

2. Temporal case, with-stops

3. Temporal case, non-stop





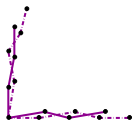


Highlights

1. Static case
   - Dijkstra, BFS, A*, ...
   - distance, path reconstruction, enumeration.

2. Temporal case, with-stops
   - $\approx$ Dijkstra, topological sort,
   - distance, path reconstruction, enumeration.

3. Temporal case, non-stop



Highlights

$\longrightarrow$ temporal data KO ☹

1. Static case $\longrightarrow$ all usual indicators OK ☺
   - Dijkstra, BFS, A*, ...
   - distance, path reconstruction, enumeration.

2. Temporal case, with-stops $\longrightarrow$ temporal OK; all usual indicators OK ☺
   - $\approx$ Dijkstra, topological sort,
   - distance, path reconstruction, enumeration.

3. Temporal case, non-stop

Highlights

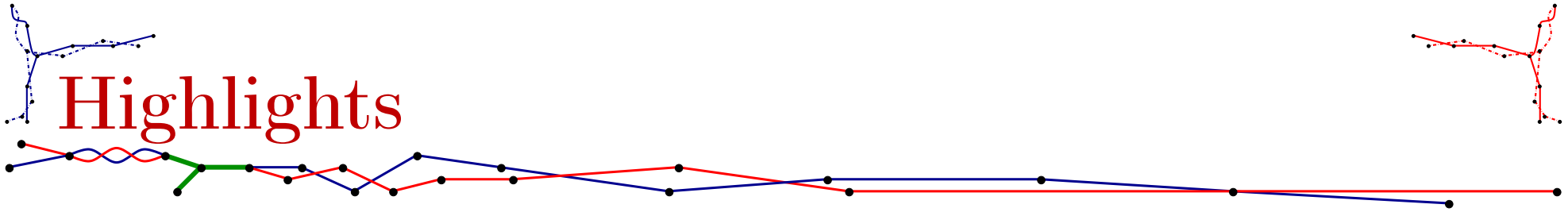
$\longrightarrow$ temporal data KO ☹

1. Static case $\longrightarrow$ all usual indicators OK ☺
   - Dijkstra, BFS, A*, ...
   - distance, path reconstruction, enumeration.

$\longrightarrow$ viral propagation KO ☹

2. Temporal case, with-stops $\longrightarrow$ temporal OK; all usual indicators OK ☺
   - $\approx$ Dijkstra, topological sort,
   - distance, path reconstruction, enumeration.

3. Temporal case, non-stop



Highlights

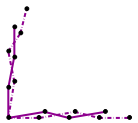
$\longrightarrow$ temporal data KO ☹

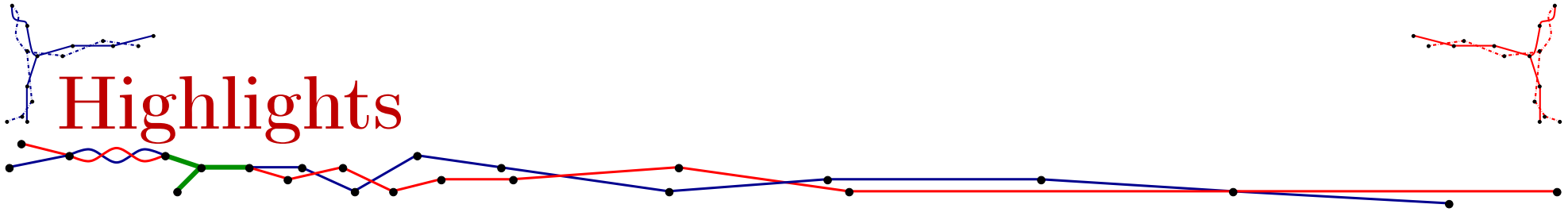
1. Static case $\longrightarrow$ all usual indicators OK ☺
   - Dijkstra, BFS, A*, ...
   - distance, path reconstruction, enumeration.

$\longrightarrow$ viral propagation KO ☹

2. Temporal case, with-stops $\longrightarrow$ temporal OK; all usual indicators OK ☺
   - $\approx$ Dijkstra, topological sort,
   - distance, path reconstruction, enumeration.

3. Temporal case, non-stop
   - paths $\neq$ walks
   - temporal walk distance, reconstruction,
   - temporal walk enumeration?





Highlights

$\longrightarrow$ temporal data KO ☹

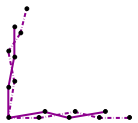
1. Static case $\longrightarrow$ all usual indicators OK ☺
   - Dijkstra, BFS, A*, ...
   - distance, path reconstruction, enumeration.

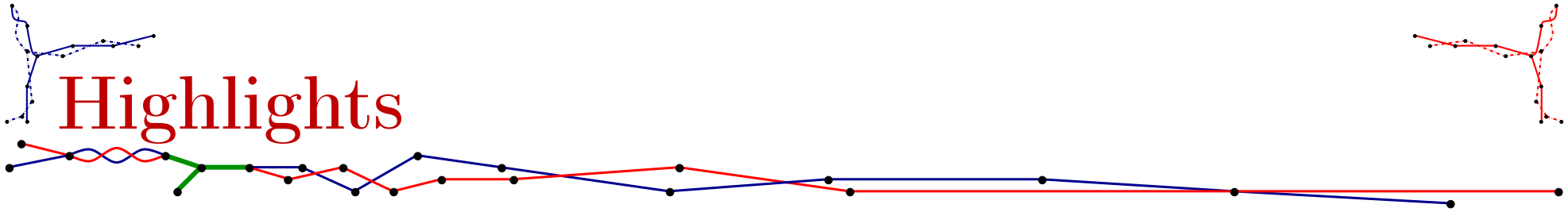
$\longrightarrow$ viral propagation KO ☹

2. Temporal case, with-stops $\longrightarrow$ temporal OK; all usual indicators OK ☺
   - $\approx$ Dijkstra, topological sort,
   - distance, path reconstruction, enumeration.

$\longrightarrow$ temporal paths NP-hard 😐

3. Temporal case, non-stop $\longrightarrow$ temporal walks OK (not enum) ☺
   - paths $\neq$ walks
   - temporal walk distance, reconstruction,
   - temporal walk enumeration?





Highlights

$\longrightarrow$ temporal data KO ☹

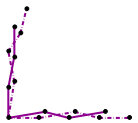
1. Static case $\longrightarrow$ all usual indicators OK ☺
   - Dijkstra, BFS, A*, ...
   - distance, path reconstruction, enumeration.

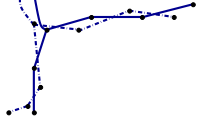
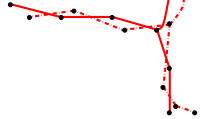
$\longrightarrow$ viral propagation KO ☹

2. Temporal case, with-stops $\longrightarrow$ temporal OK; all usual indicators OK ☺
   - $\approx$ Dijkstra, topological sort,
   - distance, path reconstruction, enumeration.

$\longrightarrow$ temporal paths NP-hard 😐

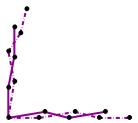
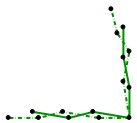
3. Temporal case, non-stop $\longrightarrow$ temporal walks OK (not enum) ☺
   - paths $\neq$ walks $\longrightarrow$ temporal walks enum. TODO ☺
   - temporal walk distance, reconstruction,
   - temporal walk enumeration?



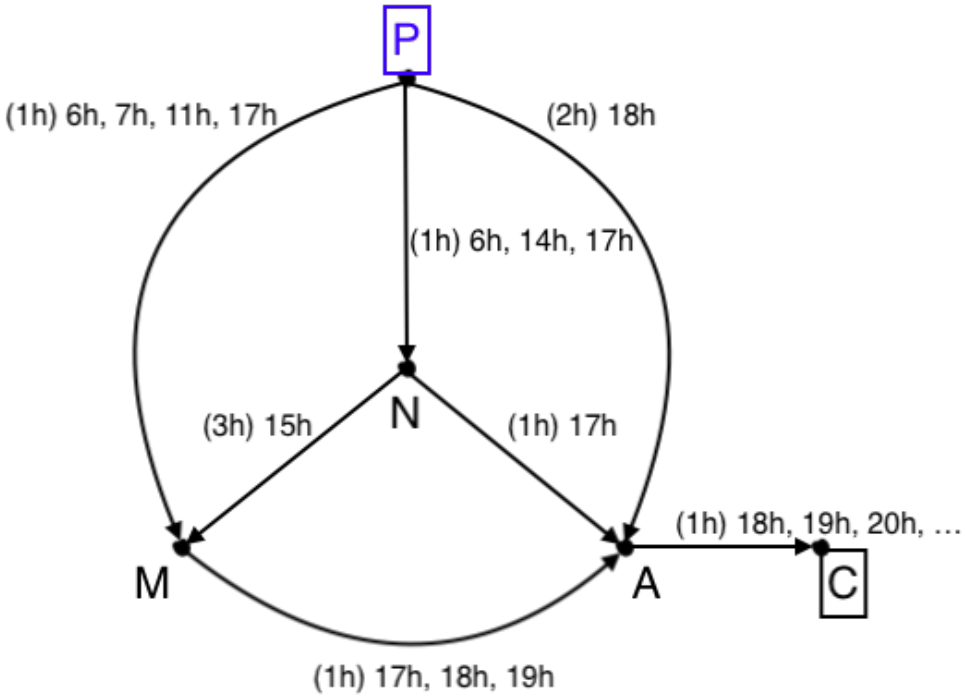
2. Temporal walks/paths with-stops

Main idea: subpath heredity

Temporal distance

Foremost arrival:



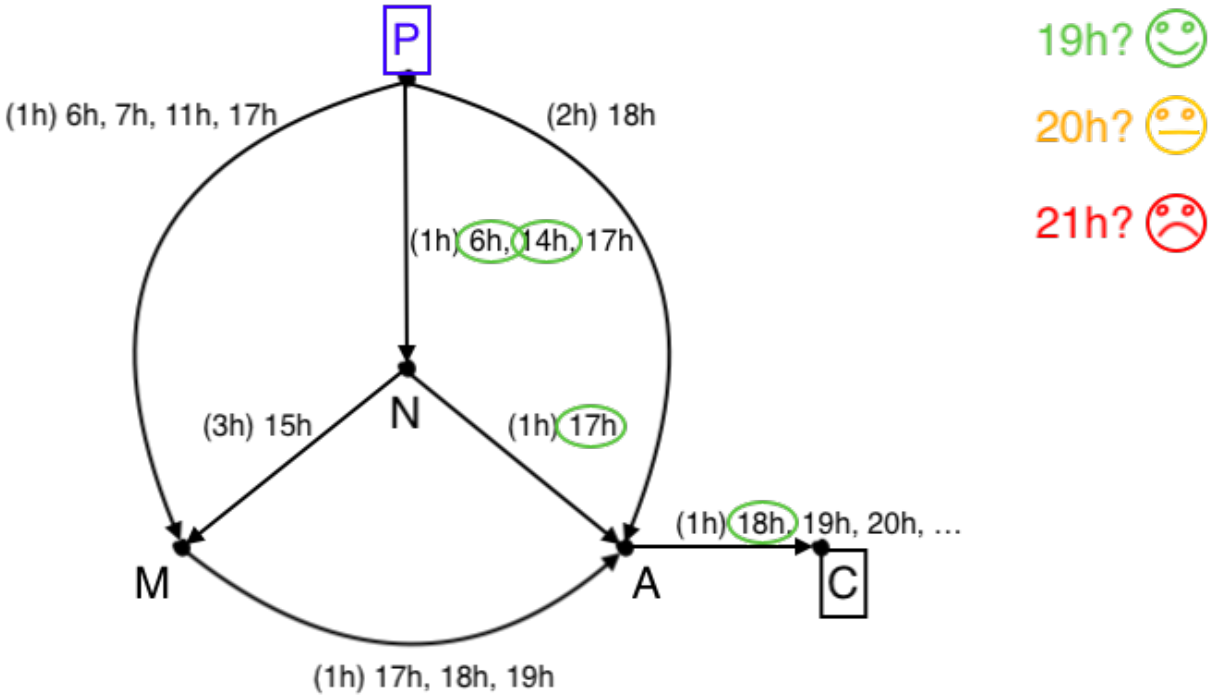
19h?

20h?

21h?

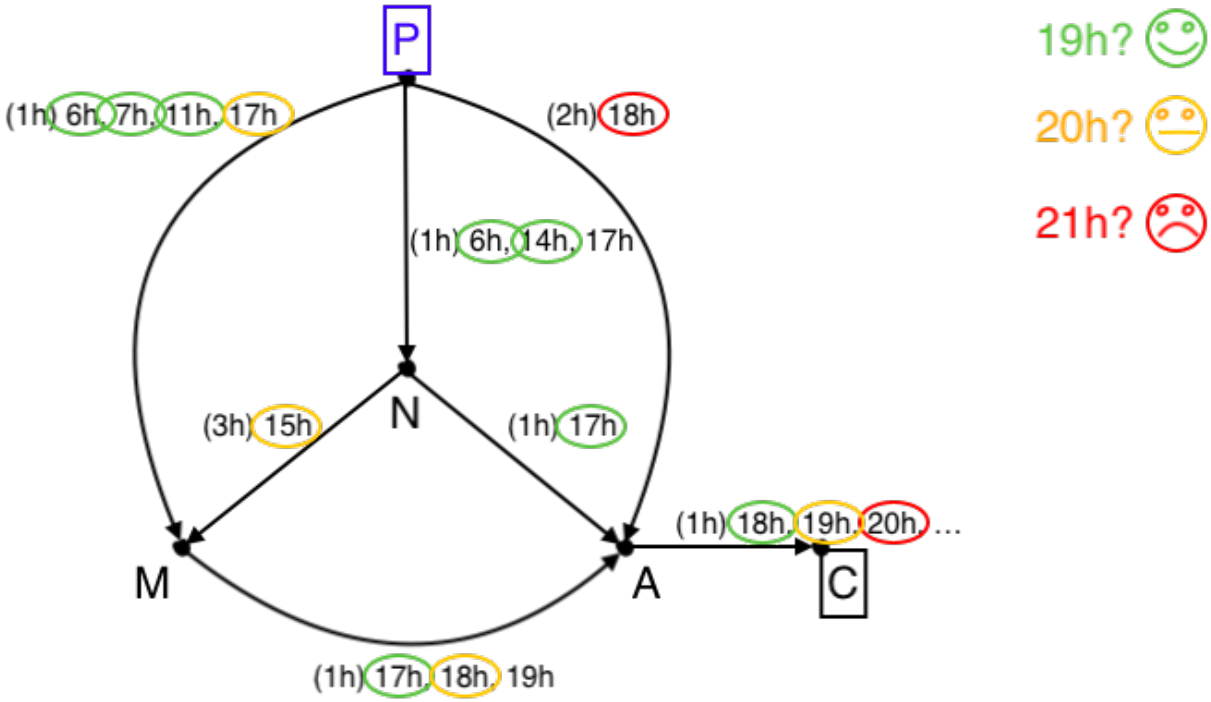
Temporal distance

Foremost arrival:



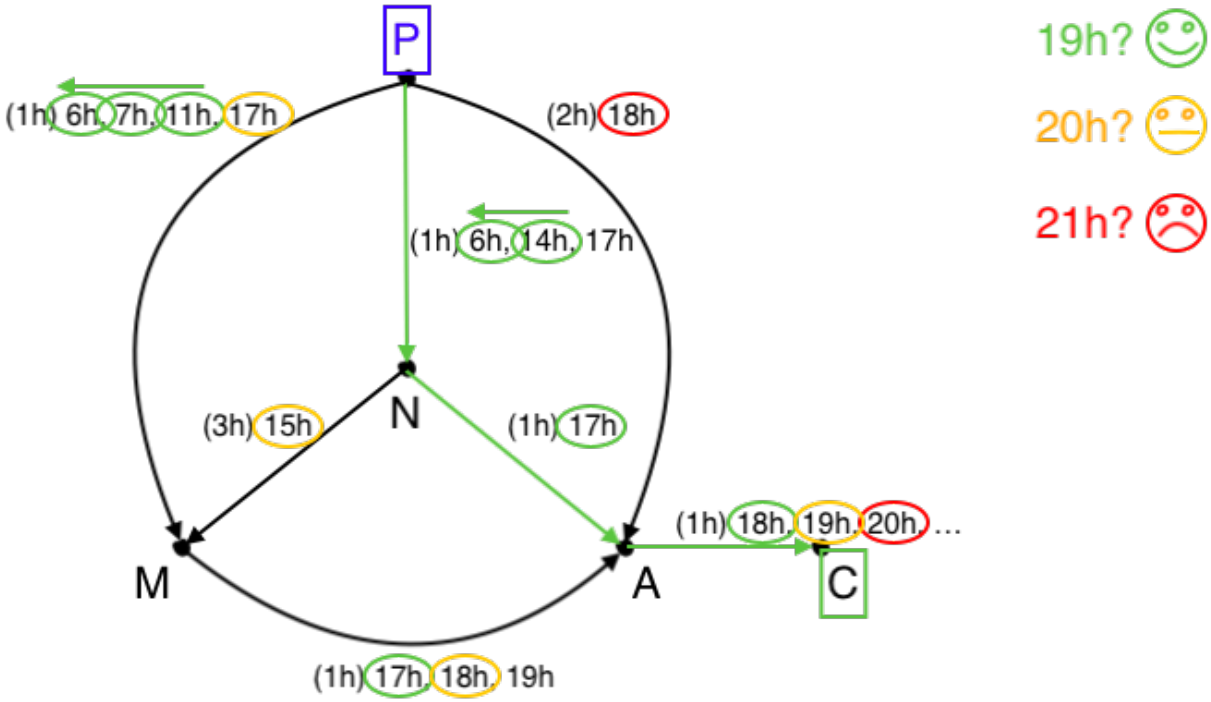
Temporal distance

Foremost arrival:



Temporal distance

Foremost arrival:



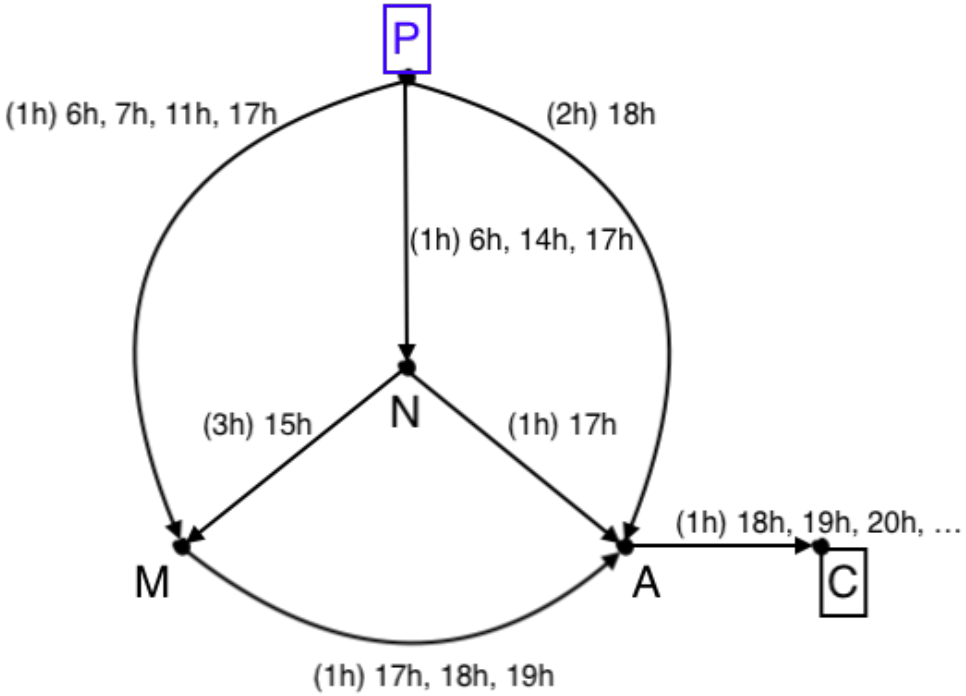
$\Rightarrow O(m \log m)$ [BX., Ferreira, Jarry, 2003]

3. Temporal case, non-stop

Main ideas: bucket sort + topological sort

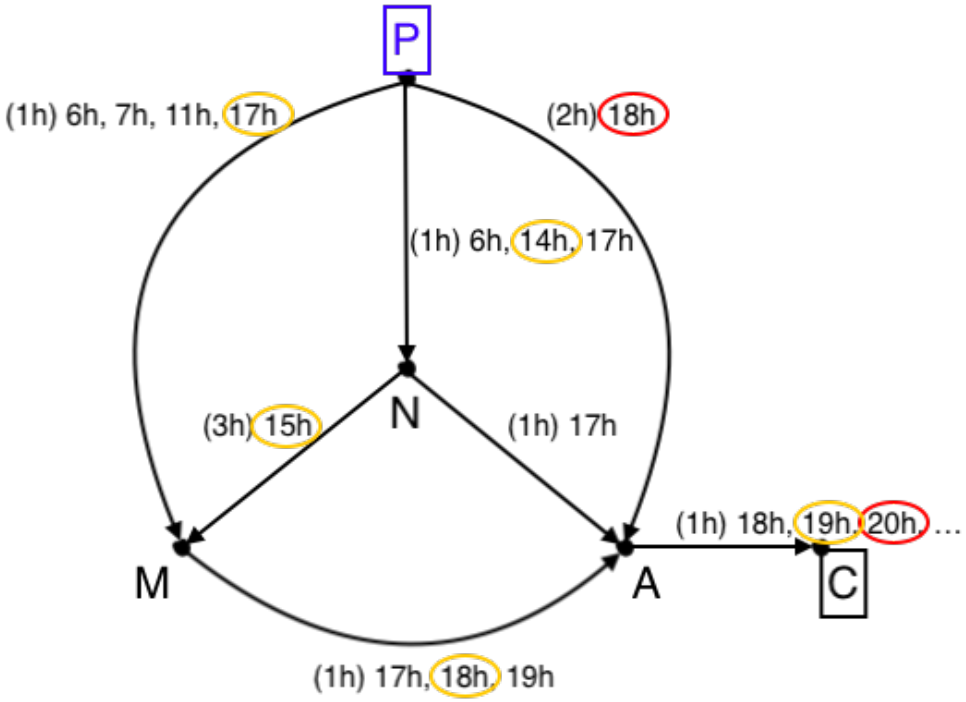
Rumor distance

Foremost arrival, non-stop:



Rumor distance

Foremost arrival, non-stop:



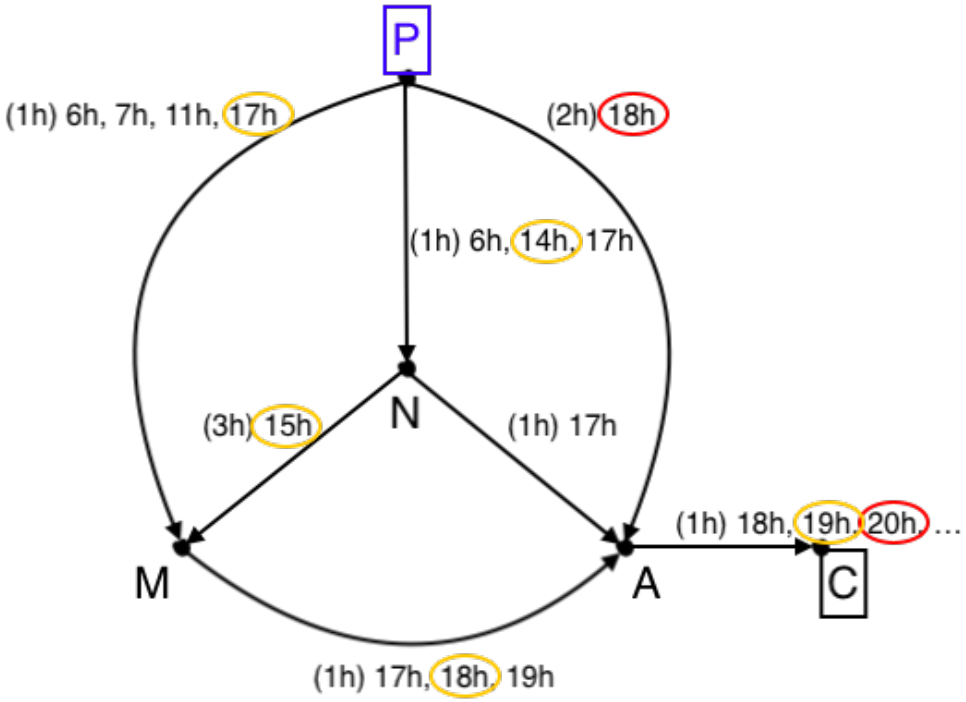
$\Rightarrow O(m \log m)$ for acyclic temporal graphs

[Himmel, Bentert, Nichterlein, Niedermeier, 2019]

$\Rightarrow$ NP-hard for the general case [Casteigts, Himmel, Molter, Zschoche, 2021]

Rumor distance

Foremost arrival, non-stop, temporal ~~path~~ walk:



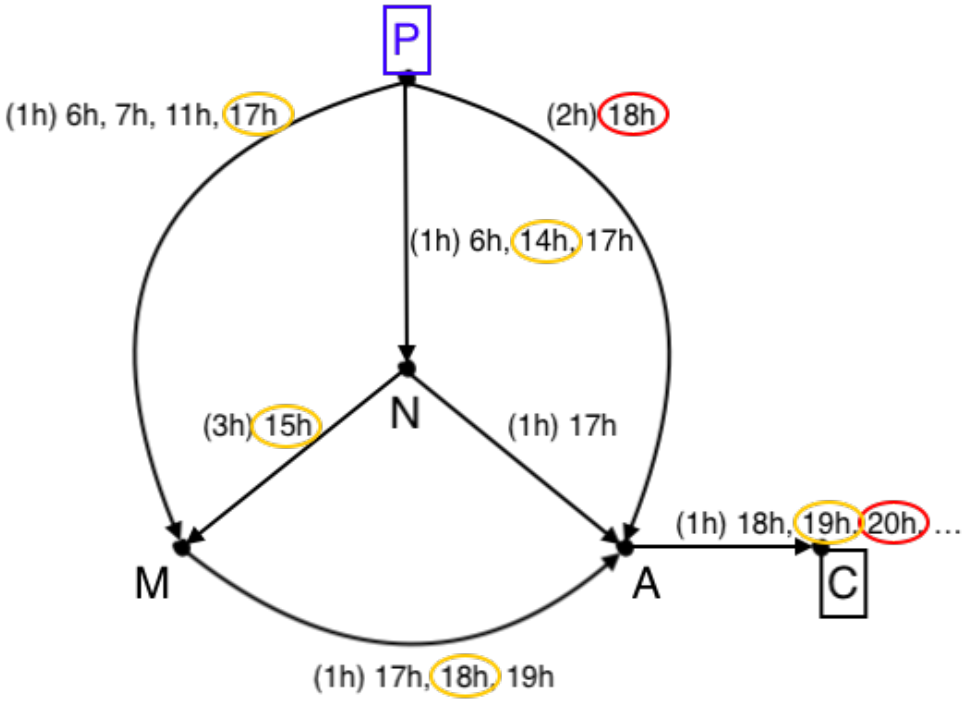
$\Rightarrow O(m \log m)$ for the general case

[Himmel, Bentert, Nichterlein, Niedermeier, 2019]

$\Rightarrow$ linear for the general case?

Rumor distance

Foremost arrival, non-stop, temporal ~~path~~ walk:



$\Rightarrow O(m \log m)$ for the general case

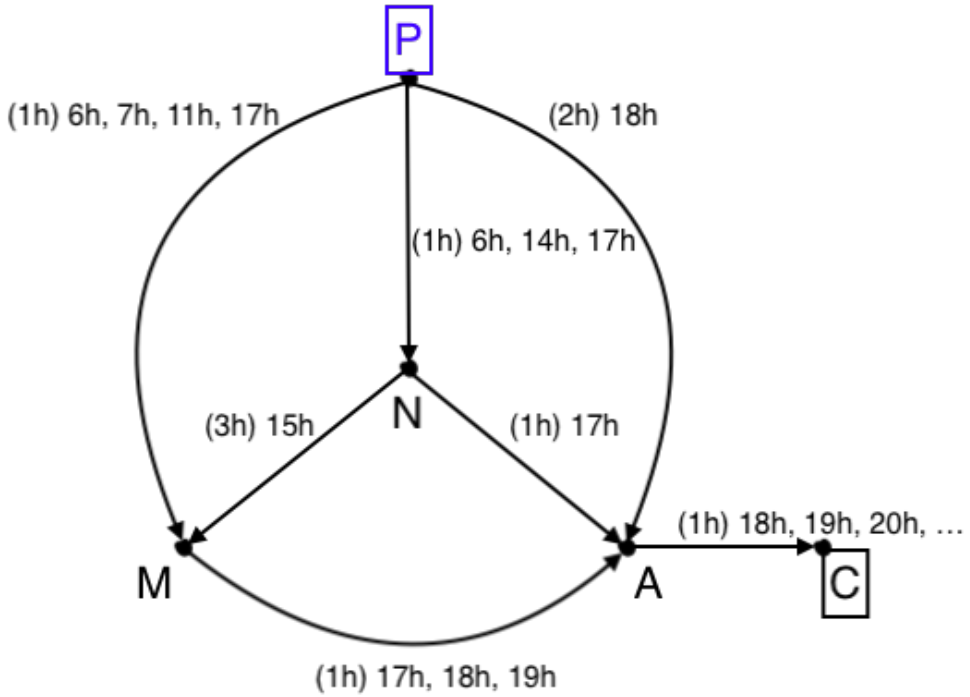
[Himmel, Bentert, Nichterlein, Niedermeier, 2019]

$\Rightarrow$ YES linear (distance) [Villacis-Llobet, BX., Potop-Butucaru, 2022]

$\not\Rightarrow$ and YES linear (walk reconstruction) [Brunelli, Viennot, 2023]

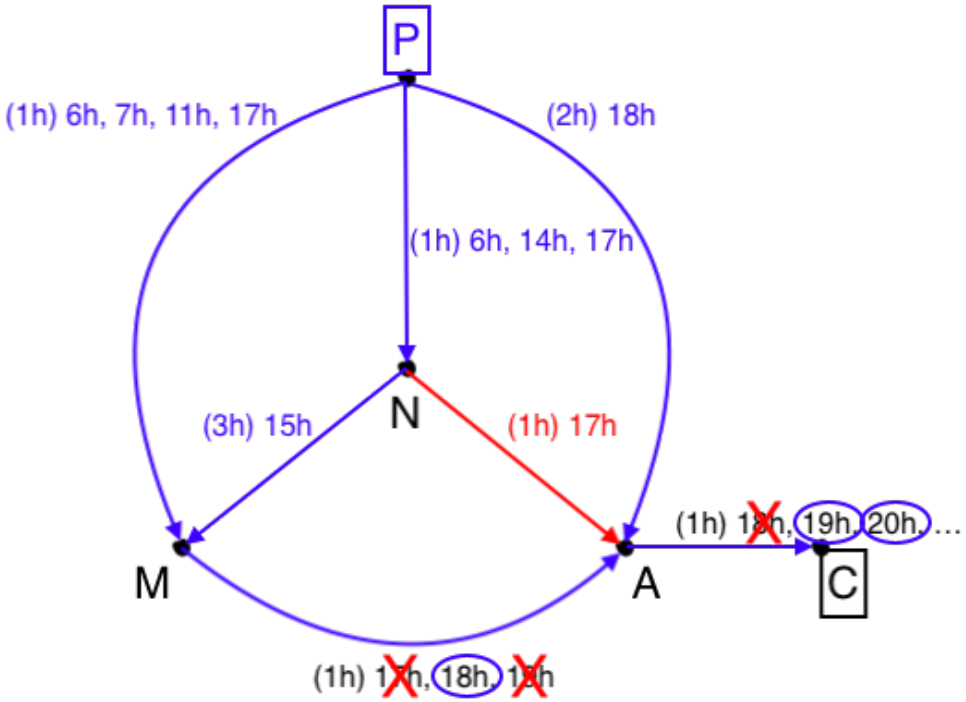
From journey to arc set

$R(s)$ = set of reachable arcs by any non-stop journey starting at s



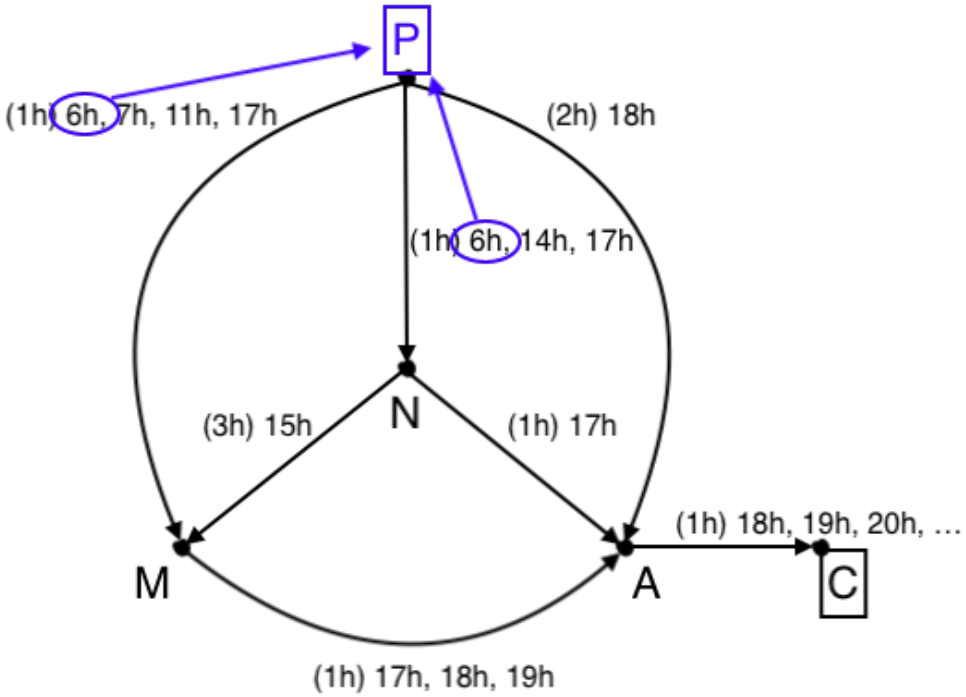
From journey to arc set

$R(s)$ = set of reachable arcs by any non-stop journey starting at s



From journey to arc set

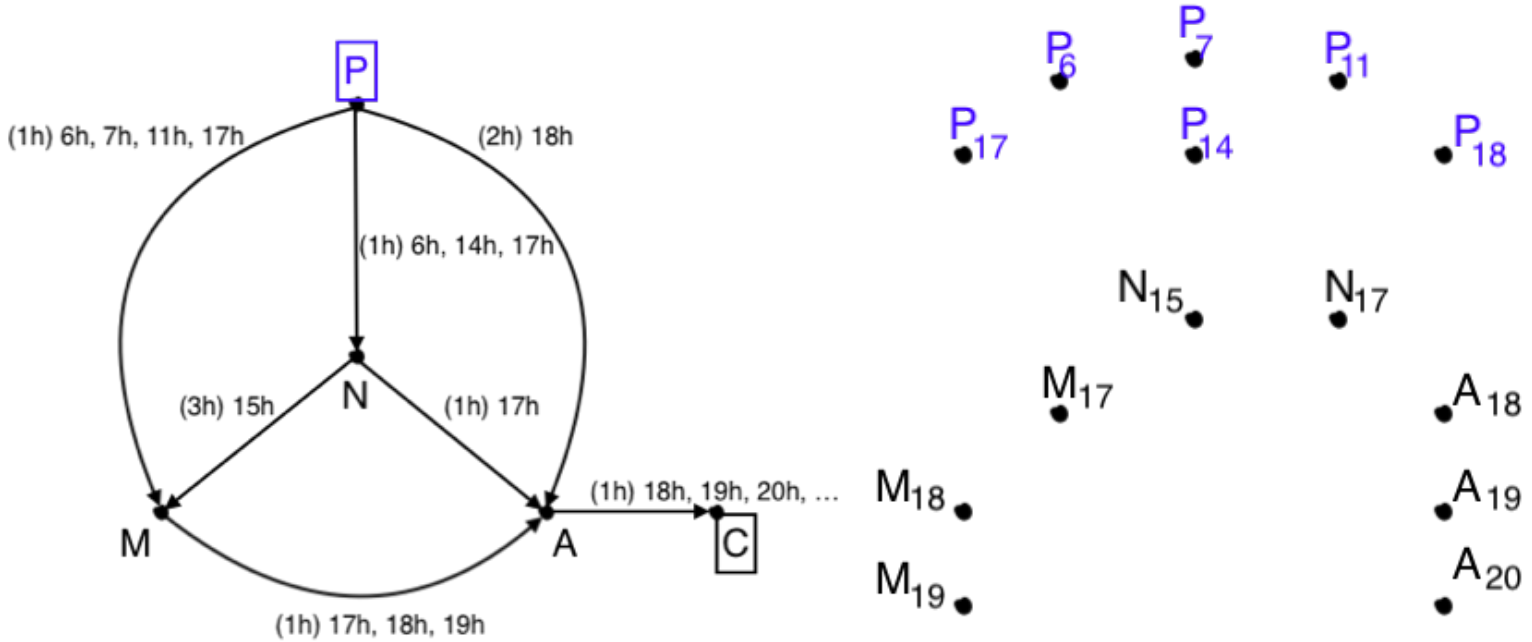
$R(s)$ = set of reachable arcs by any non-stop journey starting at s



MAIN IDEA: focus on $D(s)$ = set of departures (time) in $R(s)$

From journey to departure time

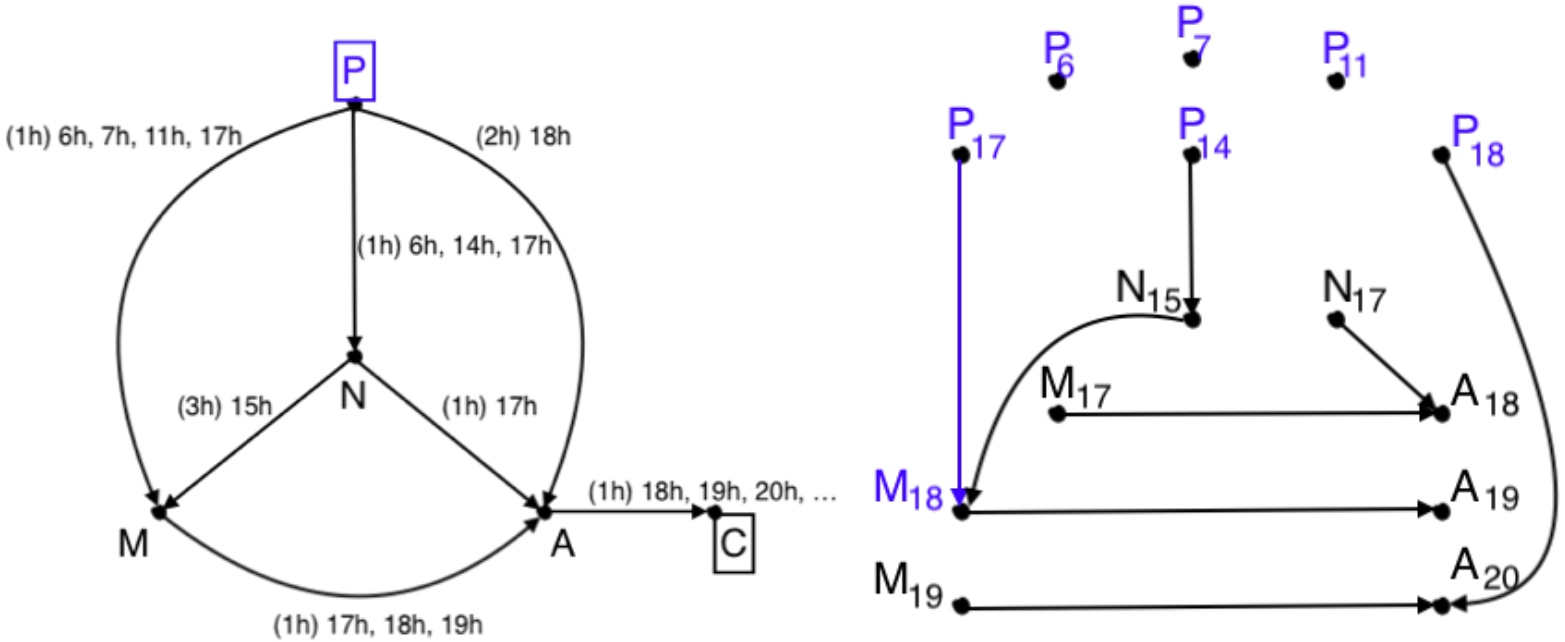
Collect all possible departures (time) into a new graph



Define $V_D = \{P_6, P_7, P_{11}, \ldots, N_{15}, N_{17}, M_{17}, M_{18}, M_{19}, \ldots\}$

From journey to departure time

Collect all possible departures (time) into a new graph

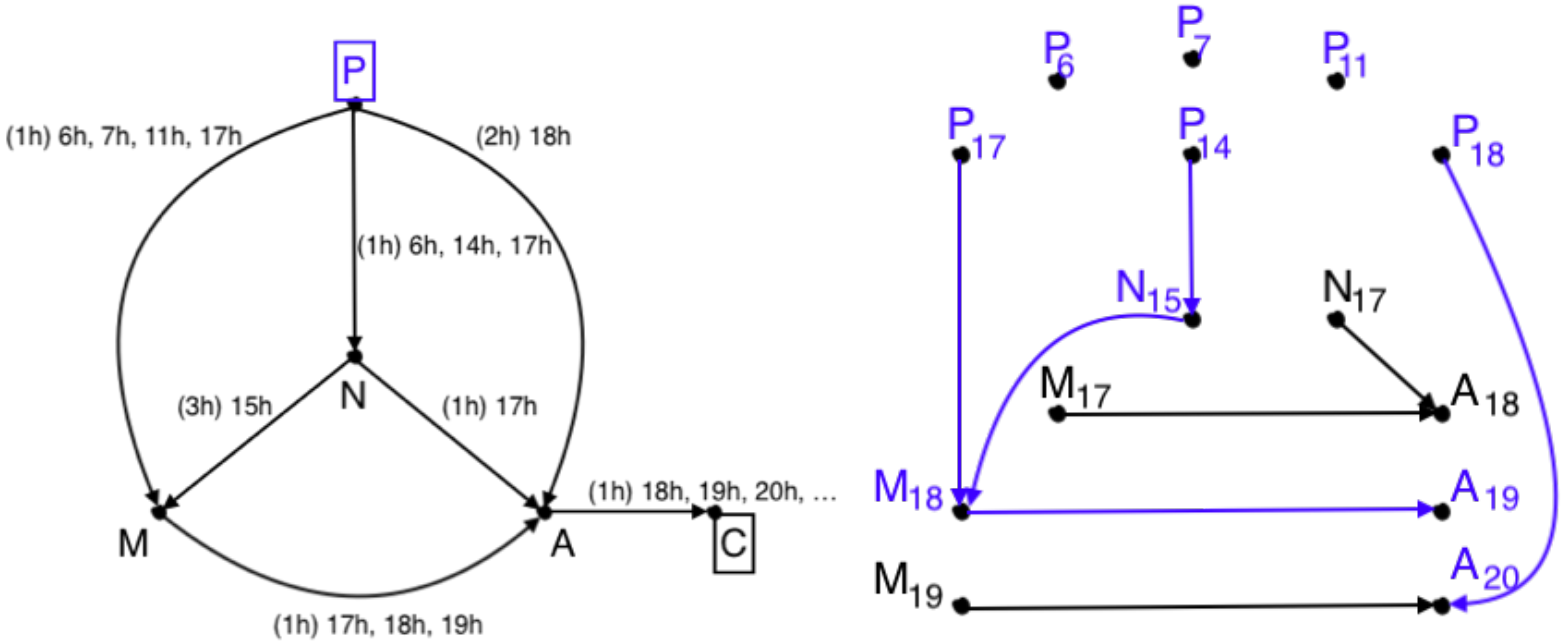


Define $V_D = \{P_6, P_7, P_{11}, \ldots, N_{15}, N_{17}, M_{17}, M_{18}, M_{19}, \ldots\}$

arc $P_{17} \to M_{18}$ means $a = (17, P, M)$ is an arc and $17 + c(a) = 18$

From journey to departure time

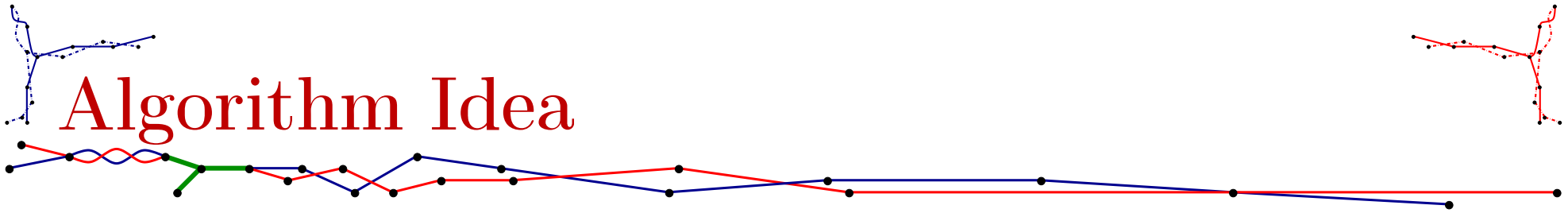
Collect all possible departures (time) into a new graph



Define $V_D = \{P_6, P_7, P_{11}, \ldots, N_{15}, N_{17}, M_{17}, M_{18}, M_{19}, \ldots\}$

Main Claim: $D(s) =$ blue vertices!

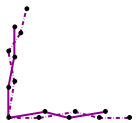
Algorithm Idea



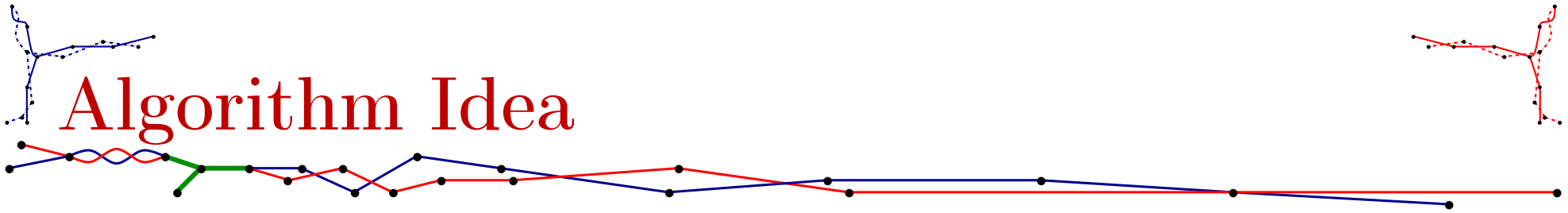
DEFINITION:

- Non-stop departure digraph $G_D = (V_D, A_D)$ is:
  - $V_D = \{v_d : \exists w \in V, (d, v, w) \in A\}$
  - $A_D = \{(v_d, w_x) : a = (d, v, w) \in A \text{ and } d + c(a) = x\}$
- Reachable sets are:
  - $R(s) = \{a : \exists \text{ non-stop journey } J = (a_1 = (\_, s, \_), a_2, a_3, \ldots, a_p = a)\}$
  - $D(s) = \{v_d : \exists a \in R(s) \text{ and } a = (d, v, \_)\}$

LEMMA: *$D(s)$ is exactly the set of vertices in G_D which are reachable from s_d, for any possible departure datetime d.*



Algorithm Idea



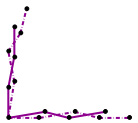
DEFINITION:

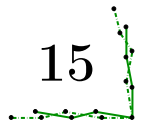
- Non-stop departure digraph $G_D = (V_D, A_D)$ is:
  - $V_D = \{v_d : \exists w \in V, (d, v, w) \in A\}$
  - $A_D = \{(v_d, w_x) : a = (d, v, w) \in A \text{ and } d + c(a) = x\}$
- Reachable sets are:
  - $R(s) = \{a : \exists \text{ non-stop journey } J = (a_1 = (\_, s, \_), a_2, a_3, \ldots, a_p = a)\}$
  - $D(s) = \{v_d : \exists a \in R(s) \text{ and } a = (d, v, \_)\}$

LEMMA: *$D(s)$ is exactly the set of vertices in G_D which are reachable from s_d, for any possible departure datetime d.*

PROPERTY: *Both $|V_D|$ and $|A_D|$ are at most equal to $|A|$.*

COROLLARY: *From G_D computing $D(s)$ is linear.*





Algorithm Idea

DEFINITION:

- ▷ Non-stop departure digraph $G_D = (V_D, A_D)$ is:
  - ◇ $V_D = \{v_d : \exists w \in V, (d, v, w) \in A\}$
  - ◇ $A_D = \{(v_d, w_x) : a = (d, v, w) \in A \text{ and } d + c(a) = x\}$

- ▷ Reachable sets are:
  - ◇ $R(s) = \{a : \exists \text{ non-stop journey } J = (a_1 = (\_, s, \_), a_2, a_3, \ldots, a_p = a)\}$
  - ◇ $D(s) = \{v_d : \exists a \in R(s) \text{ and } a = (d, v, \_)\}$

LEMMA: *$D(s)$ is exactly the set of vertices in G_D which are reachable from s_d, for any possible departure datetime d.*

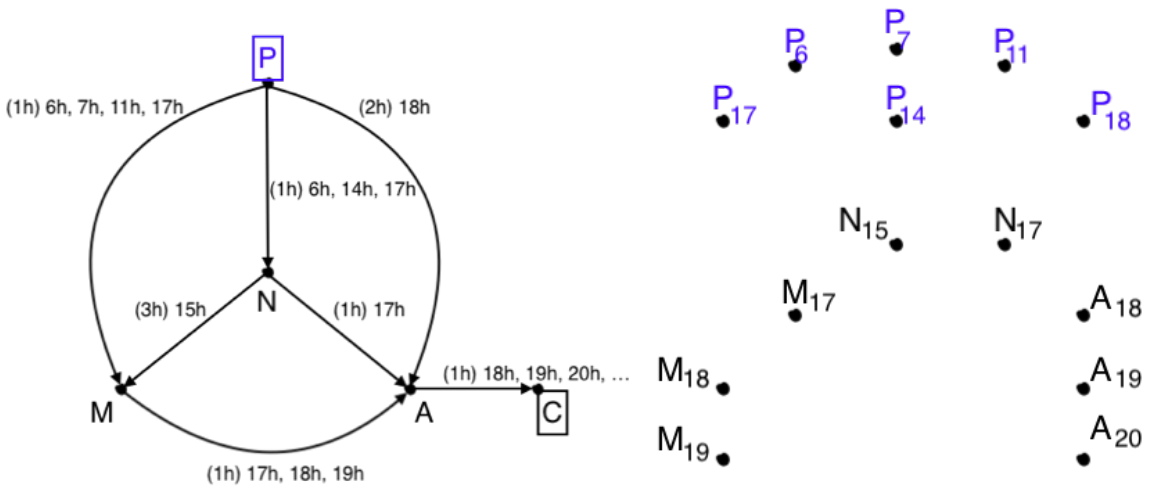
PROPERTY: *Both $|V_D|$ and $|A_D|$ are at most equal to $|A|$.*

ALGORITHM IDEA 1: *Bucket sort A and output $V_D = V_{Dep}$ in $O(|A|)$ time.*

ALGORITHM IDEA 2: *Bucket sort A again and find V_{Arr}, then construct A_D by matching $V_{Arr}(w)$ and $V_{Dep}(w)$ in order to find the right x.*

Algorithm Idea 1: V_{Arr} and V_{Dep}

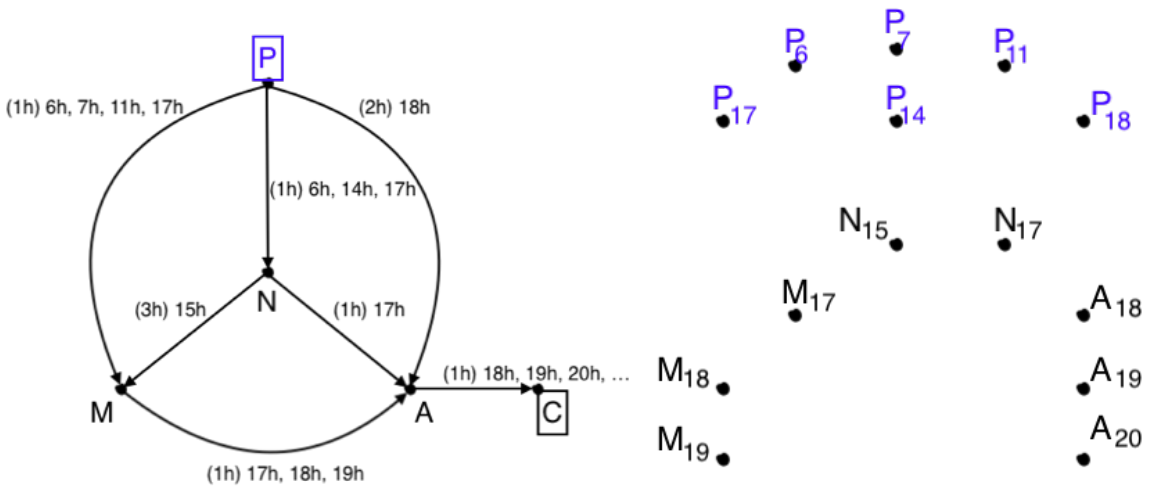
1. Initialize table `VDep` with $|V|$ entries
2. For each vertex $v \in V$ do:
3. `VDep`$[v] \leftarrow \emptyset$
4. For each arc $a = (d, v, w) \in A$ do:
5. Append d to `VDep`$[v]$ if not already present
6. Return `VDep`



Example: `VDep`$[P] = \{6, 7, 11, 14, 17, 18\}$

Algorithm Idea 1: V_{Arr} and V_{Dep}

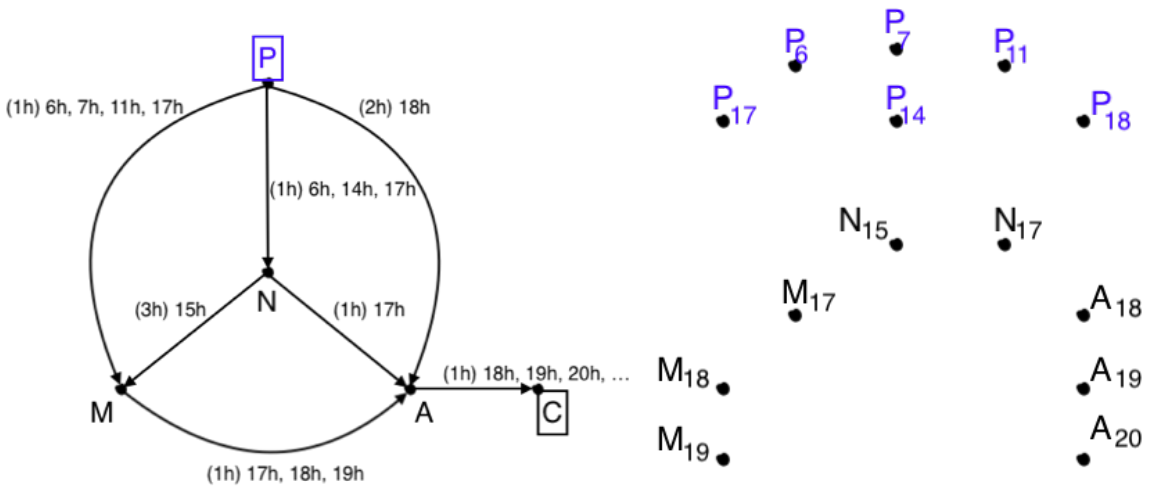
1. Initialize table `VDep` with $|V|$ entries
2. For each vertex $v \in V$ do:
3. `VDep`$[v] \leftarrow \emptyset$
4. For each arc $a = (d, v, w) \in A$ do:
5. Append d to `VDep`$[v]$ if not already present???
6. Return `VDep`



Example: `VDep`$[P] = \{6, 7, 11, 14, 17, 18\}$

Algorithm Idea 1: V_{Arr} and V_{Dep}

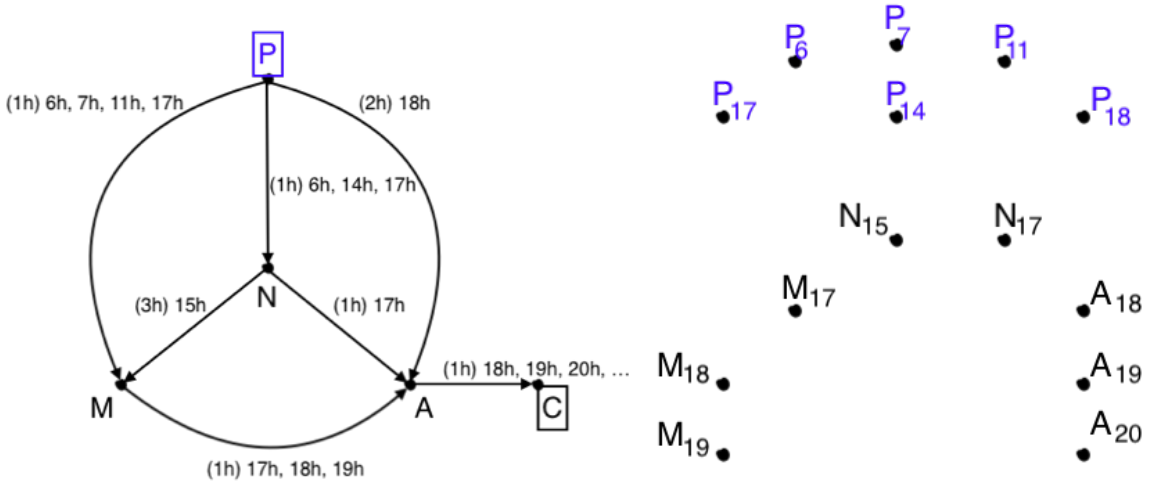
1. Initialize table `VDep` with $|V|$ entries
2. For each vertex $v \in V$ do:
3. `VDep`$[v] \leftarrow \emptyset$
4. For each arc $a = (d, v, w) \in A$ with increasing d do (req. A sorted):
5. Append d to `VDep`$[v]$ if not already present
6. Return `VDep`



Example: `VDep`$[P] = \{6, 7, 11, 14, 17, 18\}$

Algorithm Idea 1: V_{Arr} and V_{Dep}

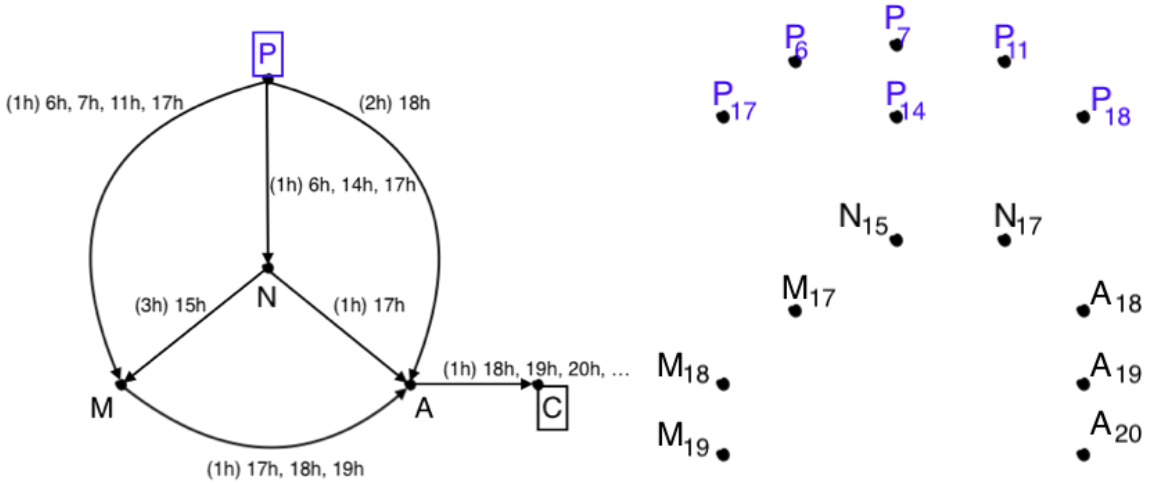
1. Initialize table `VArr` with $|V|$ entries
2. For each vertex $v \in V$ do:
3. `VArr[`v`]`$\leftarrow \emptyset$
4. For each arc $a = (d, v, w) \in A$ do:
5. Append $d + c(a)$ to `VArr[`w`]` if not already present
6. Return `VArr`



Example: `VArr[M]` $= \{7, 8, 12, 18\}$

Algorithm Idea 1: V_{Arr} and V_{Dep}

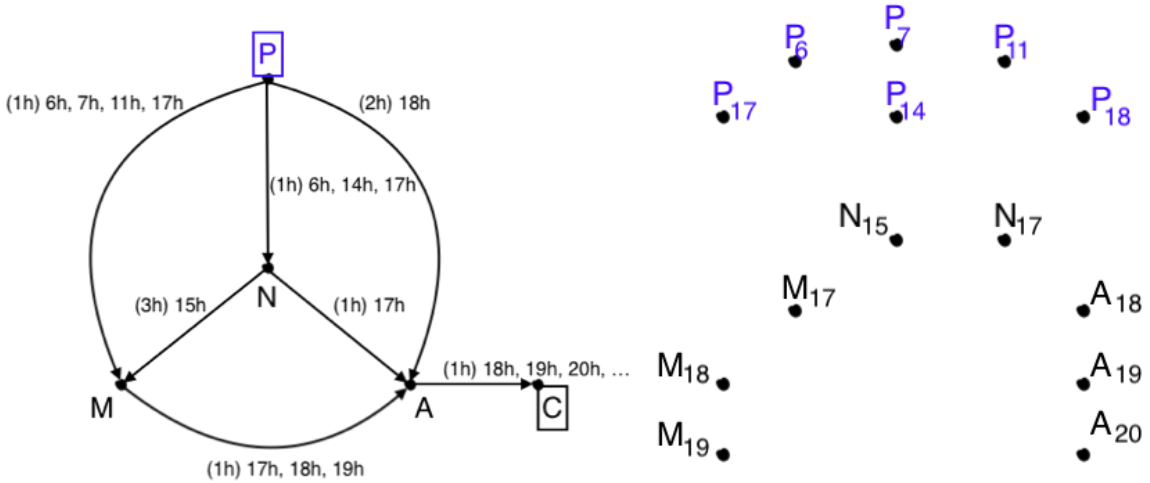
1. Initialize table `VArr` with $|V|$ entries
2. For each vertex $v \in V$ do:
3. `VArr`$[v] \leftarrow \emptyset$
4. For each arc $a = (d, v, w) \in A$ do:
5. Append $d + c(a)$ to `VArr`$[w]$ if not already present???
6. Return `VArr`



Example: `VArr`$[M] = \{7, 8, 12, 18\}$

Algorithm Idea 1: V_{Arr} and V_{Dep}

1. Initialize table `VArr` with $|V|$ entries
2. For each vertex $v \in V$ do:
3. `VArr`$[v] \leftarrow \emptyset$
4. For each arc $a = (d, v, w) \in A$ do with increasing $d + c(a)$ do:
5. Append $d + c(a)$ to `VArr`$[w]$ if not already present
6. Return `VArr`



Example: `VArr`$[M] = \{7, 8, 12, 18\}$

Algorithm Idea 0: sorting A

1. Initialize table `BatchArr` with τ entries
2. For each datetime $d \in [\![0, \tau - 1]\!]$ do:
3. `BatchArr`$[d] \leftarrow \emptyset$
4. For each arc $a = (d, v, w) \in A$ do:
5. Append a to `BatchArr`$[d + c(a)]$
6. `NewA`$\leftarrow \emptyset$
7. For each $d \in [\![0, \tau - 1]\!]$ do:
8. `NewA`$\leftarrow$ `NewA` concatenated with `BatchArr`$[d]$
9. Return `NewA`

$\Rightarrow O(\tau + |A|)$ time

Summary

LEMMA: *$D(s)$ is exactly the set of vertices in G_D which are reachable from s_d, for any possible departure datetime d.*

PROPERTY: *Both $|V_D|$ and $|A_D|$ are at most equal to $|A|$.*

ALGORITHM IDEA 0: *Bucket sort A by datetime in $O(\tau + |A|)$ time.*

ALGORITHM IDEA 1: *Bucket sort A and output $V_D = V_{Dep}$ in $O(|A|)$ time.*

ALGORITHM IDEA 1BIS: *Idem, V_{Arr}.*

ALGORITHM IDEA 2: *Construct A_D by matching $V_{Arr}(w)$ and $V_{Dep}(w)$.*

Conclusion and questions

Temporal walk distance, non-stop transit:

- ▷ linear time [Villacis-Llobet, BX., Potop-Butucaru, 2022]
- ▷ walk reconstruction linear time [Brunelli, Viennot, 2023]
- ▷ ~~journey dependent cost?~~ linear time, ≈ same algorithms
- ▷ enumeration?

Temporal path distance, non-stop transit:

- ▷ NP-hard [Casteigts, Himmel, Molter, Zschoche, 2021]
- ▷ linear time for acyclic graphs (above algorithms)
- ▷ geometric?

Both scenarios:

- ▷ hub set?